

CSCE 636 DEEP LEARNING · MODULE 12 · LECTURE NOTES

What Networks Fail At

The failure modes that do not show up in the test set.

MODULE OUTCOMES

- Explain adversarial examples and what they reveal.
- Explain shortcut learning and how to detect it.
- Explain out-of-distribution failure and overconfidence.
- State the limits of attribution methods.
- Design evaluation that exposes these failures.

1 Adversarial examples

A PERTURBATION TOO SMALL TO SEE CHANGES THE ANSWER COMPLETELY

Add a carefully chosen perturbation whose magnitude is below human perceptual threshold, and a classifier's output changes to any target class you nominate, typically with very high confidence. The two images are visually identical.

The perturbation is found by gradient ascent on the loss with respect to the *input* — the same backpropagation machinery that trained the model (Module 02), pointed at the pixels rather than the weights. **The capability is a direct consequence of the model being differentiable.**

And they transfer. An example crafted against one model frequently fools a different model, with a different architecture, trained on different data, by a different team.

Transfer is the important part. It means adversarial examples are not idiosyncratic quirks of a particular trained model but properties of the data and the learning procedure — and practically, it makes black-box attacks feasible without any access to the target model at all.

THEY MAY BE FEATURES, NOT BUGS

The intuitive reading is that the model is broken and that sufficient engineering would make it robust.

An alternative with real experimental support: the model is using genuinely predictive features that happen to be imperceptible to humans. The features are real; they are just not the ones we would use.

The experiment is striking. Construct a dataset consisting *only* of adversarially perturbed images labelled with their *target* classes — to a human, a set of mislabelled images. Train a fresh model on it. **It generalises to the real, unmodified test set.** The perturbations alone carried enough signal to learn the task.

So the perturbations are not noise; they are predictive features. The mismatch is between which predictive features a human uses and which a network uses — both are real regularities in the data, and only one of them is robust to the distribution shifts humans care about. **That is a considerably more uncomfortable finding than a bug**, because it means robustness is a constraint we must impose rather than a property we are failing to achieve.

2 Shortcut learning

Intended task	What was actually learned	How to detect it
Detect pneumonia from chest radiographs	Which hospital took the scan — identifiable from a metadata token burned into the corner, or from scanner characteristics — combined with differing base rates between sites.	Evaluate on a hospital not in the training set. Performance collapses.
Classify animals	Texture rather than shape. ImageNet-trained CNNs are substantially texture-biased, contrary to the assumption that they learn shape.	Texture–shape conflict images — a cat silhouette with elephant skin — which the model labels elephant.
Detect skin cancer	Whether a ruler appears in the frame , because dermatologists photograph lesions they already suspect alongside a scale.	Remove or add the ruler and observe the prediction change.
Answer questions about images	Dataset answer priors — answering 'two' to 'how many' and 'tennis' to 'what sport' without consulting the image.	Balanced counterfactual sets where each question has multiple answers across images.
Detect tanks in photographs	The weather on the day each set was photographed.	The apocryphal original story — probably never happened as told, and it is still the clearest statement of the failure mode.

THE TEST SET SHARES THE SHORTCUT

Training and test splits are almost always drawn from the same collection, by a random split of the same corpus. **So any spurious correlation present in the training data is present in the test data too.**

The model therefore scores well on its test set and has learned the wrong thing, and **no amount of held-out evaluation reveals it** — the held-out set is testing the same shortcut.

Detection requires a distribution shift that you construct deliberately: a different collection site, a different time period, a different device, a counterfactual edit that changes only the spurious feature, or a stress test.

This is CSCE 633 Module 12's leakage argument generalised — there, a feature carried the label; here, a feature carries the label indirectly through the collection process. **And the response is identical: be suspicious of good results, and audit what the model is actually using rather than only what it scores.**

3 Out-of-distribution failure

NETWORKS DO NOT KNOW WHAT THEY DO NOT KNOW

Presented with an input unlike anything in its training distribution, a classifier still produces a confident probability distribution over its known classes (CSCE 633 Module 10 §3) — because the softmax is constrained to sum to one and has no way to express 'none of these'.

So a model trained on ten animal species will confidently classify a photograph of a chair, a page of text, or pure noise as one of its ten species. The confidence is not merely misleading; it is frequently higher than on genuine in-distribution examples.

Detection methods exist — ensemble disagreement (Module 09), energy-based scores, Mahalanobis distance in feature space, conformal prediction (CSCE 633 Module 10 §4) — **and none is reliable across the range of shifts encountered in practice**, which is an honest summary of a large literature.

So the practical defence is to monitor the *inputs* (CSCE 633 Module 13 §1) **and to design an abstention path into the system** — a route by which a prediction can be deferred to a human or to a safer default — **rather than to trust the model's own confidence**, which is the one signal that demonstrably cannot be relied upon here.

4 Interpretability

SALIENCY MAPS ARE NOT EXPLANATIONS

A saliency map highlights which input regions the output is sensitive to. It does not say *why*, does not identify what feature was extracted, and does not indicate whether the reasoning was sound — it is a sensitivity analysis presented as a justification.

And several widely used methods fail basic sanity checks. Randomising the model's weights leaves some popular saliency maps essentially unchanged — which means those maps were reflecting properties of the *input image* (edges, salient regions) rather than anything about the model at all. The check is simple and the result was widely replicated.

A plausible-looking heatmap is persuasive and weak evidence, which is a dangerous combination — it is exactly the kind of artifact that satisfies a reviewer or a regulator without establishing anything.

Mechanistic interpretability — identifying actual circuits and features within the network, and verifying them by intervention — **is considerably more promising and far more laborious. It does not yet scale to explaining a single production decision**, which is the thing that is usually wanted, so the honest position is that explaining a network's individual output remains unsolved.

- **Test on a deliberately different distribution** — another collection site, another time period, another device, another demographic. **This single practice catches more shortcut learning than everything else combined.**

- **Construct counterfactuals:** change only the feature that should matter and verify the prediction changes; change only a feature that should not and verify it does not.
- **Stress test with corruptions** — blur, noise, compression artefacts, rotations, occlusions, weather effects. Standardised benchmarks exist (ImageNet-C and relatives) and they correlate with real deployment robustness.
- **Probe with adversarial examples even when robustness is not a stated goal** — the size of perturbation required tells you how brittle the decision boundary is, which is informative about the model regardless of whether an attacker exists.
- **And report by slice with counts** (CSCE 633 Module 12 §3), **because these failures concentrate** rather than spreading evenly — an aggregate number hides exactly the subpopulation where the shortcut does not hold. **None of this appears in a standard held-out evaluation, which is precisely why it has to be designed in deliberately.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ilyas et al. — Adversarial Examples Are Not Bugs, They Are Features (free)	The §1 reframing , with the dataset experiment. One of the more genuinely surprising results in the field.
Geirhos et al. — Shortcut Learning in Deep Neural Networks (free)	The §2 survey — the documented cases, the mechanism, and what to do about it.
Adebayo et al. — Sanity Checks for Saliency Maps (free)	The §4 result . The weight-randomisation test, and which methods fail it.
Hendrycks & Dietterich — Benchmarking Neural Network Robustness to Common Corruptions (free)	The §4 stress-test benchmark, and evidence that it predicts deployment behaviour.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Generate adversarial examples** against your model with FGSM and with PGD. Measure the perturbation size needed.
- 2 **Test transfer:** apply examples crafted against one model to a different architecture.
- 3 Plot accuracy against perturbation budget.

- 4 **Construct a deliberate shortcut** — add a small marker correlated with the label — and show the model uses it and still passes its test set.
- 5 **Then detect it** by evaluating on data without the marker.
- 6 Test a pretrained classifier on texture–shape conflict images and report the bias.
- 7 **Feed out-of-distribution inputs** and plot the confidence distribution against in-distribution inputs.
- 8 Implement an OOD detector and measure its reliability across two different kinds of shift.
- 9 **Run the saliency sanity check:** randomise the model weights and compare the saliency maps.
- 10 **Design and run a stress test suite** for your project model and report by slice.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How are adversarial examples found, and why does transfer matter?
- 2 What does the 'features not bugs' experiment show?
- 3 Give five shortcut learning cases and the detection for each.
- 4 Why does a held-out test set not reveal a shortcut?
- 5 Why are networks confidently wrong off-distribution?
- 6 Name four OOD detection methods and say how reliable they are.
- 7 What do saliency maps show, and what sanity check do several fail?
- 8 Give five evaluation practices that expose these failures.